CE 216 PROJECT

Sports Manager The Game

Design Document for "Sports Manager"

Birsu Hacıkadiroğlu 20220602039
Pelin Çinici 20240602023
Oğuz Ünlüoğlu 20240602077
Deniz Gürkan 20230602032

Table Of Contents

Table Of Contents 1

2. Project Overview 1

2.1 Core Features 1

2.2 Design Constraints 2

3. System Architecture 2

3.1 Layered Design 2

4. Abstraction Design 5

4.1 Sport Interface 5

4.2 The Player Abstract Class 6

4.3 The AbstractMatch Class Template Method Pattern 6

4.4 The MatchRules Interface 6

5. Key Design Decisions 7

5.1 Why Use an Abstract Class for Player Instead of an Interface? 7

5.2 Why Choose the Template Method for AbstractMatch? 8

5.3 Why Implement Strategy for StandingsComparator? 8

5.4 The Singleton GameSession 8

5.5 PlayerAttributeSchema 8

6. Game Flow Design 8

6.1 Application State Machine 8

6.2 The Weekly Loop 9

7. Football as the First Sport 9

7.1 Configuration 9

7.2 Player Positions and Attributes 10

7.3 Overall Rating Formula 10

7.4 Tactics 11

7.5 Match simulation model 11

8. Extensibility, Adding new sport 12

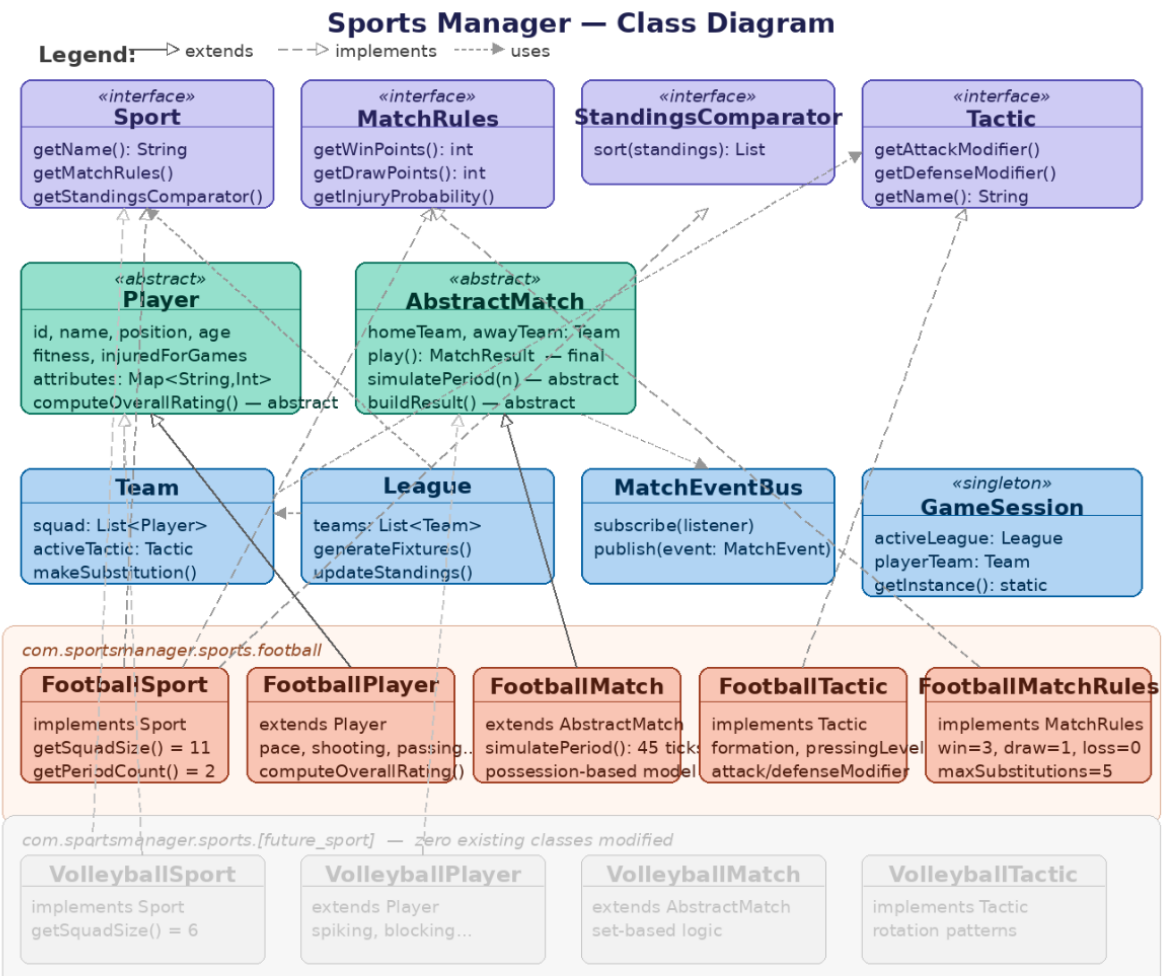
9.1 Screen Overview 13

9.2 UI and Sport Logic Separation 13

10. Data Generation and Persistence 13

10.2 Save and Load 14
11. Glossary..... 14

UML Diagram



(Generated by AI)

1. Introduction

This report outlines the software design for Sports Manager, a sports management game built with JavaFX. The player is a team manager performing the duties of a club: from organizing

a squad, through setting tactics and training sessions, up to playing matches against AI-controlled teams week after week.

The main design problem of this project is that the application should be able to handle an unlimited number of different kinds of sports without obtaining any prior knowledge about the sports. Football will be the first sport to be implemented, yet the overall design should also be capable of accommodating the addition of sports such as volleyball, basketball, or others without the need to make any changes to the existing code.

This paper records the design choices that have been decided upon prior to starting the coding work: which abstractions will be introduced, how the different layers will interact, and how the features of each individual sport will be separated from the rest of the program. The only reason why implementation details are mentioned is to give a clearer picture of the design.

2. Project Overview

2.1 Core Features

The game will have these features:

- Sport selection when starting a new game, football initially, other sports will be implemented later.
- When a new league is created the application will automatically generate teams, players, and coaches.
- The full schedule of matches in a double round-robin league will be played, that is every team plays every other team twice.
- The game loop will be weekly: training during the week and matches on the weekend.
- The player will decide on the pre-match lineup and tactics.
- The match will be simulated period by period and substitutions may be made during breaks.
- A player injury system, players who are injured will be unavailable for a certain number of matches.
- League table with sport-specific tiebreakers.
- Long-term playing, after the champion is determined, the season is reset and continued.

2.2 Design Constraints

There are several constraints that affect every design decision in this document:

- Sport-specific logic should be completely isolated. The main program framework should not include any football-related code. All sport behavior should be behind interfaces.

- A new sport can only be added by creating new classes, no changes to existing classes will be allowed. This is a direct use of the Open/Closed Principle.

- The UI should be based on abstractions only. No JavaFX controller should import any class from the sports package.

3. System Architecture

3.1 Layered Design

In order to build the whole application, we will use a layered design where each of the layers will have its own specific responsibility. The most important rule that will govern these layers is the direction of the dependency: higher layers are dependent on lower layers only though interfaces; lower layers never depend on higher ones.

Layer	Package	Responsibility	Depends On
Presentation	com.sportsmanager.ui	JavaFX screens and controllers. Responsible for all user interaction.	Core interfaces only
Application	com.sportsmanager.app	Manages the flow of the game: session state, week advancement, match running, save/load.	Core interfaces only
Core Framework	com.sportsmanager.core	Provides all interfaces, abstract classes, and domain models not related to a particular sport.	Nothing above
Sport Modules	com.sportsmanager.sports.*	Realizations of all core interfaces for a particular sport.	Core interfaces only

DESIGN RULE	*The sports.football package may import from core. The ui and app packages may import from core. Nothing in ui or app may import from sports.football or any other sport package.*

3.2 Package Structure

The planned package and file structure is as follows:

Package / File	Type	Purpose
com.sportsmanager.core	Package	All interfaces, abstract classes, and shared domain models
Sport.java	Interface	Root contract for any sport — configuration, rules, factories
Player.java	Abstract Class	Common player state; sport-specific rating left abstract
AbstractMatch.java	Abstract Class	Template method for match flow; period simulation left abstract
Team.java	Class	Squad, lineup, coaches, active tactic
League.java	Class	Teams, fixtures, standings map
MatchRules.java	Interface	Points system, max substitutions, injury probability
StandingsComparator.java	Interface	Pluggable table-sorting strategy
Tactic.java	Interface	Attack/defense modifiers, formation parameters
MatchEvent.java	Abstract Class	Goal, injury, substitution, period-end event types
MatchEventBus.java	Class	Observer hub — publishes events to UI subscribers
com.sportsmanager.sports.football	Package	Football-specific implementations
FootballSport.java	Class	Implements Sport; defines positions, periods, schema
FootballPlayer.java	Class	Extends Player; position-weighted overall rating
FootballMatch.java	Class	Extends AbstractMatch; possession-based simulation
FootballTactic.java	Class	Implements Tactic; formation + pressing level

FootballMatchRules.java	Class	Implements MatchRules; 3/1/0 points, 5 subs
FootballStandingsComparator.java	Class	Implements StandingsComparator; GD tiebreaker
FootballDataLoader.java	Class	Generates random teams and players for football
com.sportsmanager.app	Package	Game orchestration — no sport-specific imports
GameSession.java	Singleton	Active league, player team, current week
GenerationService.java	Class	Creates leagues via sport factories
LeagueOrchestrator.java	Class	Advances weeks, triggers AI matches
MatchOrchestrator.java	Class	Runs matches synchronously or on a background thread
TrainingOrchestrator.java	Class	Applies training sessions to player attributes
SaveLoadService.java	Class	Serializes/deserializes GameSession to .smg files
SportRegistry.java	Singleton	Maps sport names to their Sport implementations
com.sportsmanager.ui	Package	JavaFX screens — depends only on core and app
Main.java	Application	Entry point; initializes stage via SceneNavigator
SceneNavigator.java	Class	Central scene-switching utility
MainMenuScreen.java	Class	Title screen: new game, load game, quit
LeagueScreen.java	Class	Weekly overview: table, fixtures, training
PreMatchScreen.java	Class	Lineup picker, tactic selector
MatchScreen.java	Class	Live event log, period scores, substitution panel

4. Abstraction Design

This part is the core of our first milestone. It specifies each interface and abstract class that will act as stable borders between the main framework and the sport-specific modules.

4.1 Sport Interface

Sport being the main abstraction, is the single point of entry through which the application gets to know a sport's entire configuration. UI will request SportRegistry upon startup for a list of sports; upon selection of a sport, that particular Sport implementation will be the source of all rules of the game.

EXTENSIBILITY	*The way to add a new sport is for a developer to make a class that implements the Sport interface and then return correct values for different methods. Away from this, the whole application league, standings, fixture generation, the UI does not have to be changed at all.*

4.2 The Player Abstract Class

Player is going to be an abstract class and not an interface because all players in different sports have common player state: unique IDs, names, ages, fitness levels, injury counters, and a map of named attributes. What varies in sports is the way the overall rating is derived from those attributes, a goalkeeper's rating calculation is totally different from that of a striker, and a volleyball player's rating calculation is different from both.

By declaring computeOverallRating() abstract, we ensure that each sport has its own calculation method while at the same time inheriting all common facilities. The attribute map is kept generic on purpose (Map<String, Integer>) so that any sport may define any attributes it requires without affecting the Player class.

4.3 The AbstractMatch Class Template Method Pattern

Match simulation will rely on the Template Method design pattern. AbstractMatch specifies the immutable sequence of steps for conducting a match: a pre-match method, a period loop with post-period methods after each period, and a final result construction. The actual content of each period, i.e., what happens inside it, remains abstract and will be implemented by the specific sport match classes.

With this approach, the code responsible for the match flow exists in one and only one location. Introducing a new sport just requires implementing a new simulatePeriod() method; the complete structure remains unchanged.

PATTERN	*Template Method, AbstractMatch.play() is the fixed sequence. simulatePeriod(), getPeriodCount(), and buildResult() are the variable elements that each sport's concrete match class implements.*

4.4 The MatchRules Interface

Instead of hardcoding point values or substitution limits directly into the League or standings logic, we'll pull all those rules from the MatchRules interface. The League class will call sport.getMatchRules() just once and then use that returned object whenever it needs to hand out points or check limits. This setup means that switching from football's 3/1/0 points system to headball's 2/1/0 system only requires a new MatchRules implementation—no changes needed in the League class itself.

4.5 The Standings Comparator

Interface — Strategy Pattern Tiebreaker rules can vary quite a bit from one sport to another. In football, if teams are tied on points, we break the tie using goal difference, then goals scored, and finally head-to-head results. Other sports might use win percentage, sets won, or even a coin toss. By defining StandingsComparator as an interface and injecting it into the League through the Sport object, we're applying the Strategy pattern: the sorting algorithm becomes a pluggable strategy instead of being hardcoded. The League class keeps a Map of LeagueRecord objects and calls standingsComparator.sort(standings) whenever it needs to display the table. It doesn't need to know or care about how the sorting happens.

4.6 The Tactic Interface

Every sport will have its own tactical formations with various parameters. For instance, football tactics include a pressing level, an attack modifier, and a defense modifier linked to specific formations like 4-3-3 or 4-4-2. A future sport might have completely different tactical parameters. The Tactic interface lays out the basic contract that the match engine requires: an attack modifier and a defense modifier. Additional parameters can be accessed through a generic Map<String, Object> getParameters() method, which each sport can fill with any relevant tactical data.

4.7 The MatchEvent System

Observer Pattern The match simulation will run on a background thread to keep the UI responsive. The simulation needs to communicate what's happening to the UI layer without interrupting the user experience.

PATTERN	*Observer — MatchEventBus decouples the simulation engine from the UI. The engine only knows about MatchEvent; the UI only knows about MatchEventBus. Neither knows about the other.*

5. Key Design Decisions

5.1 Why Use an Abstract Class for Player Instead of an Interface?

Theoretically, writing Player as a Java interface would seem a perfect idea. But at the same time, all the sports have a lot in common when it comes to player state and behavior. For example, things such as UUID generation, the map of attributes, fitness levels, injury times, and availability checks are all common. Instead of repeating that shared code in each and every player class of different sports, we can simply put it in an abstract class. What is more, only the method that should be different computeOverallRating() is defined as abstract so that different sports can supply different implementations.

5.2 Why Choose the Template Method for AbstractMatch?

A more traditional way would be to bundle the match simulation logic in one single method of a concrete class and then use if/else or switch statements to differentiate between sports. However, such a design would go against the Open/Closed Principle: every time a new sport is added, the match class would be modified. Thanks to AbstractMatch, the structure stays the same while the matching sport is fully simulated by the subclasses. Therefore, the main part of the application is not modified if a new sport is added.

5.3 Why Implement Strategy for StandingsComparator?

Tiebreakers can get quite complex and are different enough for a number of sports so that a single flag cannot be used for their control. By converting it to a Strategy object returned by the Sport interface, tiebreaker rules of each sport are entirely encapsulated and can be independently tested. The getTable() method of the League class simply delegates to the comparator, thus remaining void of any sport-specific logic.

5.4 The Singleton GameSession

GameSession will be implemented as a Singleton. This Singleton is responsible for managing the active league, the player's selection, and the current week number. Opening a new game session will be possible by resetting the singleton through the reset() method. Although Singletons are sometimes considered an anti-pattern, this one fits the concept entirely since only one game session can exist at a time. Besides, the only other option that is passing session state as a parameter through every layer would have led to unnecessary coupling.

5.5 PlayerAttributeSchema

We could simply have some hardcoded constants here and there in the code to check player attributes but that wouldn't be very good practice. Instead, we will use a PlayerAttributeSchema class. So each sport will generate a schema defining which attributes can be set and what the values should be between. The schema will become a player helper during player generation by ensuring the validity of the initial values and during training, it will monitor attributes to prevent exiting a limit. In this way, the framework itself will become more unified.

6. Game Flow Design

6.1 Application State Machine

The UI of the application consists of a set of well-defined states where each one of them corresponds to a separate screen and the application moves between them automatically or by player action or game event such as a season end.

6.2 The Weekly Loop

It is the LeagueOrchestrator that controls the sequence of steps for an in-game week:

- First, it shows the player the games scheduled for the current week and the entire season's schedule.

- Then the player is shown the league standings and the player's squad status including which players are injured, the fitness levels, etc.

- The player can arrange and conduct a training session for his/her side.

- The player gets to the PRE_MATCH flow in case there is a scheduled game for the week.

- The AI-controlled fixtures for that week will be simulated automatically by the LeagueOrchestrator.

- Standings will be updated for all completed fixtures, and injury counters will be adjusted.

- The week counter will move forward, and if all weeks are completed, the season will conclude.

6.3 Match Flow Detail

The match is structured so that players can step in between periods. The AbstractMatch.play() method runs each period through simulatePeriod(), and after each one, postPeriodHook() publishes a PeriodEndEvent. The UI listens for this event, pauses the simulation, and displays the current score, giving the manager a chance to make substitutions or adjust tactics. Once the player confirms their choices, the simulation picks up again for the next period. This design keeps the UI and simulation neatly separated, ensuring that the match engine never directly calls any UI methods.

7. Football as the First Sport

7.1 Configuration

Football will be the first concrete implementation of the Sport interface. Its configuration will be:

Parameter	Value
Sport Name	Football
Squad Size (Starters)	11
Substitutes on Bench	4 (up to 5 substitutions per match)
Periods per Match	2 (two halves)
Points: Win / Draw / Loss	3 / 1 / 0

7.2 Player Positions and Attributes

Football players will have seven attributes, each with a defined range in the PlayerAttributeSchema. The available positions will be GK, CB, LB, RB, CM, LM, RM, and CF.

Attribute	Range	Most Important For
Pace	40–99	Full-backs, wingers, strikers
Shooting	20–99	Strikers, attacking midfielders
Passing	30–99	Central midfielders, full-backs
Dribbling	30–99	Wingers, attacking midfielders
Defending	20–99	Centre-backs, defensive midfielders
Heading	20–99	Centre-backs, strikers
Stamina	40–99	All positions

7.3 Overall Rating Formula

FootballPlayer overrides computeOverallRating() to use a weighted formula for the specific position. This way ratings are position-specific instead of just being generic. For instance, the rating of a centre-back will mostly come from Defending and Heading while the rating of a striker will mostly come from Shooting and Pace. Goalkeepers will have their own formula that emphasizes the availability of their specialized attributes.

7.4 Tactics

There will be only 5 tactical formations available in the initial football version(Just for now as planned). Each tactic has attack and defense multipliers which are used in the match simulation, plus a pressing level that is a further play style modifier.

Tactic Name	Formation	Attack Modifier	Defense Modifier	Character
4-3-3 Attack	4-3-3	1.30	0.90	High-risk, high-reward
4-4-2 Balanced	4-4-2	1.00	1.00	Neutral — no tradeoffs

5-3-2 Defensive	5-3-2	0.80	1.30	Prioritises not conceding
3-5-2 Pressing	3-5-2	1.10	0.95	Aggressive mid-press
4-2-3-1 Counter	4-2-3-1	1.05	1.10	Fast transitions

7.5 Match simulation model

The model of the football match simulation is a tick-based possession system. A half is divided into 45 ticks each representing one minute. On every tick the simulation will:

-
- Calculate each team's side possession probability on the basis of the ratio of their attacking strengths, which is further modified by the current tactic.
-
- Find out if the team in possession is able to create a shooting chance (base probability at present: 5% per tick).
-
- When there is a shot, the goal-scoring probability is calculated by comparing the attacker's strength with the defender's one (conversion ratio at present: 35%).
-
- Each team's active players is going to get checked for injury, though the chances are quite low.

This model is still pretty basic because it is the very first one done. It is capable of producing realistic score distributions (usually around 0, 4 goals per match) while also being computationally very fast.

Besides, the simulation logic of the match itself resides exclusively in FootballMatch.simulatePeriod() so it is very easy to adjust it in the future without having to do anything in other classes.

8. Extensibility, Adding new sport

The entire architecture is planned with only one goal in mind: adding a new sport should involve only writing new code, never changing old code. Below are the steps that a developer will follow to add volleyball as a second sport, for example.

Step	Action	Classes Involved
1	Create package com.sportsmanager.sports.volleyball	—
2	Implement Sport → VolleyballSport	Define positions (S, OH, MB, L, RS, OPP), 5 periods (sets), attribute schema
3	Extend Player → VolleyballPlayer	Define computeOverallRating() using volleyball attributes (setting, spiking, blocking, serving, digging)
4	Extend AbstractMatch → VolleyballMatch	Implement simulatePeriod() for set-based volleyball rules; getPeriodCount() returns 5
5	Implement Tactic → VolleyballTactic	Define rotational tactics with appropriate attack/defense modifiers
6	Implement MatchRules → VolleyballMatchRules	Define points system; may need a different injury model
7	Implement StandingsComparator → VolleyballComparator	Sort by sets won, then points ratio
8	Create VolleyballDataLoader	Generate random volleyball teams and players
9	Register in SportRegistry	SportRegistry.register(new VolleyballSport())

RESULT

After these nine steps, the application's sport selection screen will automatically show Volleyball as an option. No existing class — not League, not Team, not any UI controller — will have been modified.

9. User Interface Design

9.1 Screen Overview

We plan to develop the UI using JavaFX, with the help of FXML for layout, and a controller class for each screen. All controllers will run on the service classes at the app-layer level (GameSession, LeagueOrchestrator, MatchOrchestrator) and also on the core model types (Team, Player, Fixture, LeagueRecord). However, no controller will import any class from the sports package.

9.2 UI and Sport Logic Separation

The Pre-Match screen requires presenting the player with the selection of tactics available per their sport. It will invoke team.getActiveTactic() and fill the dropdown with tactic names returned by sport.getTacticNames(). The screen/view is oblivious of whether the returned strings denote football formations or volleyball rotations it just showcases the strings that the Sport implementation returns. This is the epitome of the principle of programming to an interface showing in the UI layer.

In a similar vein, the player attribute window will fetch attribute names and their corresponding values from player.getAttributes(), a generic Map<String, Integer>. The UI will display all the attributes present in the map. Football players will have attributes like Pace, Shooting, Passing, etc. Volleyball players will have attributes such as Setting, Spiking, Blocking, etc. The screen logic is indifferent to these changes.

10. Data Generation and Persistence

10.1 Random Generation

Once a league has been initiated, the teams and players are created in a completely automated fashion. Player names will be chosen from pairs of first and last names that encompass a wide array of nationalities. Team names will be fetched from a set list. Logos will be chosen at random from a group of included image files.

Player attribute values will be generated using position-aware random ranges defined by the sport's PlayerAttributeSchema. A goalkeeper's initial shooting attribute will be drawn from a smaller range than that of a striker; a centre-back's pace will come from a lower range of values than that of a winger. This provides a credible diversity of player types throughout the generated league.

10.2 Save and Load

The game state will be converted to JSON format using the Gson library and stored in a file with the.smg extension. The GameSession singleton, that holds the entire league, all the teams, all the players, and the current week, will be the main object to be serialized. When loading, the GameSession singleton will be substituted with the deserialized version and the user interface will immediately display the game as it was restored.

11. Glossary

Term	Definition
Sport Interface	The base Java interface which every sport has to implement. It serves as the primary reference storing all the configuration details of a sport.
Sport Module	An independent unit (for example, com.sportsmanager.sports.football) that brings to life the full set of main interfaces of a sport.
PlayerAttributeSchema	A map between attribute names and their valid numeric ranges, which is set separately for each sport.
Fixture	A match that is set up to take place between two teams during a particular week of the league.
Lineup	The chosen players who will start the game as well as the substitutes for a certain match.
Tactic	A named configuration of the team strategy that changes attack and defense multipliers during simulation

Period	A segment of time within the match (e.g. half, quarter, or set) at the end of which the manager has an opportunity to make substitutions..
MatchEventBus	The communicating channel which distributes simulation events from the match engine to the user interface layer in an observer pattern.
StandingsComparator	A strategy object that can be switched out which decides the way a sport's league table is ordered and how tiebreakers are settled.
GameSession	The globally unique application singleton encapsulating all ongoing game states: the league, the player team, and the present week.
.smg file	Sports Manager Game the proprietary save file extension of the application.